MOSAIC: A Multi-Agent System for Segment-Aware Marketing Campaign Generation

Team 13 • Progress Report 1 • 22 August 2026

Project Title	MOSAIC — A Multi-Agent System for Segment-Aware Marketing Campaign Generation
Date of Report	22 August 2026
Reporting Period	11 August 2026 – 22 August 2026 (first fortnight; no previous report)
Team Members	Ong Qing Quan, Agustinus Benyamin Prasetyo
Report Number	1 of 4

1. Summary of Work Done

This is our first report, so everything below covers work done since the proposal was accepted. The goal for these two weeks was to get the groundwork in place — repository, database, model access and the agent contracts — rather than to build any of the nine pipeline stages properly. We deliberately kept the scope narrow so that the orchestration work starting next sprint does not sit on top of an unstable base.

Environment and repository setup

- Set up the monorepo with a FastAPI backend and a React 18 + Vite + TypeScript frontend. Both are started from a single `start.ps1` script so either of us can bring the whole thing up on a clean machine.
- Created the Supabase project and wrote the first migration (`0001_init.sql`) covering `campaign`, `customer_profile`, `segment_brief`, `offer_catalogue`, `brand_guideline_pack` and the append-only `ledger_entry` table.
- Pinned the Python dependencies (FastAPI, httpx, LangGraph, scikit-learn, HDBSCAN) and got a matching install working on both our machines.

Model access layer

- Wrote the OpenRouter wrapper (`backend/app/openrouter_client.py`) that every agent will call. It injects the API key, tags each call with stage / campaign id / prompt version, records token usage, and refuses the call if the campaign is already over its budget ceiling.
- Confirmed with live calls that both models we planned on are reachable through one key: `openai/gpt-5.6-luna` for text and `google/gemini-3.1-flash-image` for images. Also confirmed the text model honours `response_format: json_schema`, which is what lets us keep every agent output typed instead of parsing free text.
- Built a small cost meter on top of the wrapper so we can see spend per stage. Two weeks of testing has cost us roughly US$6, which suggests the per-campaign budget we assumed in the proposal is not unrealistic.

Data and content

- Wrote the synthetic customer generator (`synthetic_data.py`) and produced a first 5,000-row dataset with demographics, purchase history, engagement behaviour and a short free-text preference field. It is seeded, so the same run reproduces the same data.
- Drafted the fictional brand (“Northbank Living”) used throughout the demo: tone-of-voice notes, a small guideline pack and about twenty offers in the offer catalogue. Having real offer rows matters because the Guardian stage later has to check that every claim in an email traces back to one of them.

Architecture and contracts

- Wrote the build plan document that turns the proposal into concrete components: stage-by-stage responsibilities, which stages call a model and which are plain deterministic code, and what each stage reads and writes.
- Defined the JSON schema for each artifact that moves between agents (segment brief, creative brief, copy version, guardian verdict). Agents never call each other directly — everything passes through the shared store and is validated at the boundary.
- Put up a skeleton Campaign Console with the navigation shell and placeholder screens, so we have somewhere to hang the real views as each stage lands.

Against the 15-task plan in the proposal, tasks 1 to 3 are done and task 4 (orchestration backbone) has been started but is not finished. That is roughly a fifth of the build, which is about where we expected to be at this point.

2. Effort Expended

Hours logged by each member since the start of the project (no previous report to measure from).

Work item	Ong Qing Quan	Agustinus Benyamin Prasetyo	Total
Repository, tooling and local run setup	2.5	1.0	3.5
Supabase schema and first migration	1.0	3.0	4.0
OpenRouter wrapper, cost meter, budget ceiling	4.0	0.5	4.5
Synthetic dataset generator and 5,000-row seed	0.5	3.5	4.0
Northbank Living brand pack and offer catalogue	0.5	2.5	3.0
Build plan document and agent JSON contracts	3.0	2.0	5.0
Console skeleton and navigation shell	3.0	0.5	3.5
Model access troubleshooting and spikes	2.0	1.5	3.5
Team meetings and coordination	1.5	1.5	3.0
Total	**18.0**	**16.0**	**34.0**

Split of responsibilities: Ong Qing Quan is taking the orchestrator, the model access layer and the frontend console; Agustinus Benyamin Prasetyo is taking data, database schema, brand and offer content, and the evaluation side. The architecture work was done together because both of us needed to agree on the contracts before splitting off.

3. Problems Encountered

Embeddings through OpenRouter are not confirmed

Our segmentation plan assumes we can embed the free-text preference field through the same OpenRouter key we use for everything else. Chat and image endpoints work, but the embeddings endpoint has been inconsistent across providers in our testing. If it does not work we will either call an embedding provider directly (which means a second key and second billing surface) or run a local sentence-transformer.

HDBSCAN install on Windows

HDBSCAN needed C++ build tools on one of our machines and cost us most of an afternoon. It is working now, but it makes us wary about the marker or a teammate not being able to run the project. We are considering pinning a wheel or containerising the backend.

Named-approver identity is still undecided

The proposal requires approvals and BLOCK overrides to be attributable to a named person. A full auth system feels heavy for a two-person academic demo, and a simple “pick your name” selector would cover the audit requirement. We plan to go with the light option unless advised otherwise — confirmation from the lecturers would help.

Judging realism of the synthetic data

The generator produces clean data, and clean data clusters too neatly. That risks making the segmentation stage look better than it is. Benyamin is adding noise, missing fields and some overlap between groups so the clustering has something real to fail at.

4. Plan for the Next 2 Weeks

Target for 5 September: a campaign can be created in the console, run through segmentation, and produce segment briefs that are written to the database and visible on screen.

Task	Owner	Target date
Finish the LangGraph orchestration backbone (stage sequencing, state passing, error handling)	Ong Qing Quan	27 Aug
Write every stage transition to the append-only decision ledger	Ong Qing Quan	29 Aug
Feature engineering: encode structured attributes and join with the text embedding	Agustinus Benyamin Prasetyo	28 Aug
HDBSCAN clustering, cluster-stability check and confidence score	Agustinus Benyamin Prasetyo	1 Sep
Segment rationale generation (text model) with typed JSON output	Agustinus Benyamin Prasetyo	2 Sep
Segment Dashboard screen wired to real segment briefs	Ong Qing Quan	3 Sep
Settle the embeddings question and record the decision	Both	26 Aug
Decide polling vs. streaming for run progress in the console	Both	29 Aug
Dry run over a 500-row subset and record cost and latency per stage	Both	4 Sep

Creative Director and Copywriter (tasks 6 and 7) stay out of scope for this sprint. We would rather have one stage working end to end with the ledger and the database behind it than three stages half done.